1. **Implement Tic –Tac –Toe Game**
   Implement "Tic-tac-Toe" in general without using any standard algorithm.

   Code:

```
# Tic-Tac-Toe Game


# Human = X
# Computer = O

board = [' ' for _ in range(9)]


def display_board():

    print()
    print(" ", board[0], "|", board[1], "|", board[2])
    print("---+---+---")
    print(" ", board[3], "|", board[4], "|", board[5])
    print("---+---+---")
    print(" ", board[6], "|", board[7], "|", board[8])
    print()


def check_winner(player):

    winning_positions = [
        (0, 1, 2),
        (3, 4, 5),
        (6, 7, 8),
        (0, 3, 6),
        (1, 4, 7),
        (2, 5, 8),
        (0, 4, 8),
        (2, 4, 6)
    ]

    for a, b, c in winning_positions:

        if board[a] == player and board[b] == player and board[c] ==
player:
            return True

    return False


def board_full():

    return ' ' not in board


def computer_move():

    for i in range(9):
```

```
        if board[i] == ' ':
            board[i] = 'O'
            return


print("TIC-TAC-TOE")
print("Human = X")
print("Computer = O")


while True:

    display_board()

    # Human Move
    while True:

        position = int(input("Enter your position (1-9): "))

        if position < 1 or position > 9:
            print("Enter a number between 1 and 9.")

        elif board[position - 1] != ' ':
            print("Position already occupied.")

        else:
            board[position - 1] = 'X'
            break

    # Check Human Win
    if check_winner('X'):
        display_board()
        print("Human wins!")
        break

    # Check Draw
    if board_full():
        display_board()
        print("Game Draw!")
        break

    # Computer Move
    computer_move()

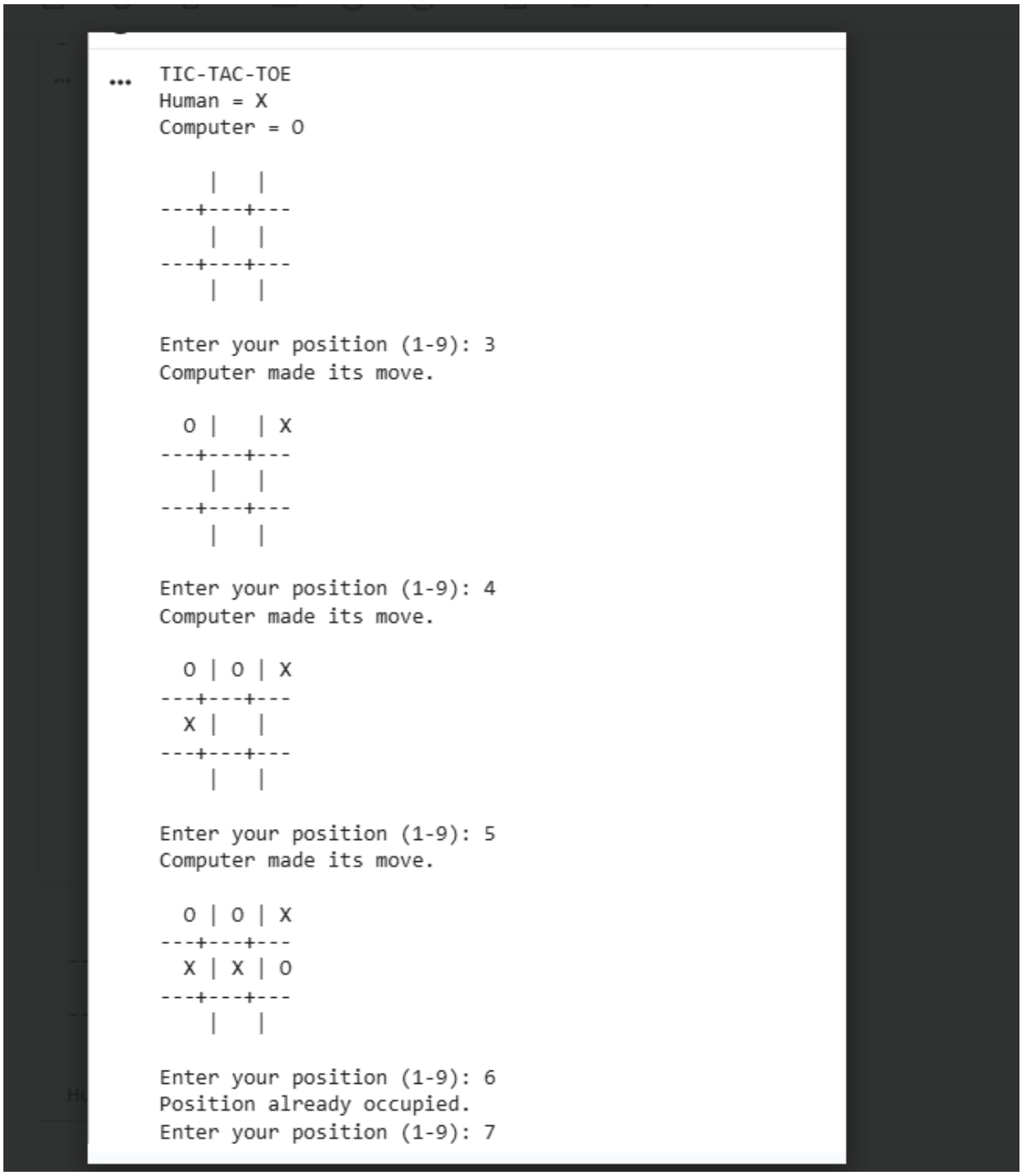
    print("Computer made its move.")

    # Check Computer Win
    if check_winner('O'):
        display_board()
        print("Computer wins!")
        break

    # Check Draw
    if board_full():
        display_board()
```

```
        print("Game Draw!")
        break
```

out put



```
Enter your position (1-9): 7

  O | O | X
 ---+---+---
  X | X | O
 ---+---+---
  X |   |

Human wins!
```

2. **Implement vacuum cleaner agent**
   Implement "Vacuum Cleaner Agent" using the concept of "Simple Reflex agent and Goal based" architecture

Code:

```
# Vacuum Cleaner - Simple Reflex Agent

room = {
    "A": "Dirty",
    "B": "Dirty"
}

location = "A"


def simple_reflex_agent(location, room):

    if room[location] == "Dirty":
        return "Suck"

    elif location == "A":
        return "Move Right"

    else:
        return "Move Left"


while True:

    print("\nLocation:", location)
    print("Room A:", room["A"])
    print("Room B:", room["B"])

    action = simple_reflex_agent(location, room)
```

```
    print("Action:", action)

    if action == "Suck":
        room[location] = "Clean"

    elif action == "Move Right":
        location = "B"

    elif action == "Move Left":
        location = "A"

    if room["A"] == "Clean" and room["B"] == "Clean":
        print("All rooms are clean!")
        break
```

Output:

